# MAYUR VEER

veer.m@northeastern.edu | 857-351-9373 | LinkedIn | GitHub | Portfolio

## EDUCATION

**Northeastern University, Boston, MA** **Expected Dec 2026**

Master of Science in Software Engineering Systems

**Relevant courses**: System Design in Age of AI, Network Structures and Cloud Computing, Program Structure and Algorithms, Concepts of Object-Oriented Design, Theory & Practical Applications of AI Gen Model

## PROFESSIONAL EXPERIENCE

**Cybersenz LLC, Remote** **Jan 2026 – May 2026**

*Software Engineering Intern*

- Containerized a **6-service stack** (FastAPI, React, email agent, Postgres, Redis, Vault) into a **single-command Docker Compose deploy** with in-container Alembic migrations and auto-restart on all 6 services for self-healing recovery
- Hardened Azure production to **zero secrets on disk** by injecting **Key Vault** secrets via **Managed Identity** at startup, closing public access to Postgres and Vault, and terminating TLS at nginx
- Shipped production clinic software with **50+ JWT-secured FastAPI endpoints** and React dashboards for FHIR-style patient registration, role-based access and audit logging, integrated with **PrognoCIS EHR** and **HMAC-signed webhooks**

**Ernst & Young LLP, Mumbai, INDIA** **Sept 2021 – Aug 2024**

*Team Lead – Software Engineer*

- Accelerated invoice processing by **17x** by reengineering data script with Pandas optimizations, enabling team to process **25,000 invoices in 3 minutes** which took 1 hour for 1,800 previously, eliminating processing bottleneck for monthly billing operations and reducing infrastructure costs by **40%**
- Delivered **90% manual effort reduction** for Formula 1 by automating Arabic invoice generation with context-aware PDF translation, processing **500+ invoices weekly** at **99.8% accuracy**
- Reduced application response time by **60%** through React optimization (useCallback, Context API, localStorage), improving UX for **100+ daily users** across modules and decreasing bounce rate by **35%**
- Cut page load latency by **50%** by implementing Angular lifecycle optimization with conditional data refresh using ngOnChanges, reducing client-side utilization by **60%**
- Led **4-person engineering team** in delivering full-stack application on schedule, establishing code review standards that reduced production bugs by **40%** and improved deployment velocity by **4x**

## PROJECTS

**Self-Hosted PaaS with Fault-Tolerant Scheduler | Go, Python, FastAPI, Postgres, K8s, Terraform**

- Built a Heroku-style PaaS where git push triggers Kaniko build → deploy → verify via a **Go Kubernetes operator** (CRDs, server-side apply) with **automatic rollback** on failed health checks, fronted by a FastAPI control plane
- Ported a from-scratch distributed job scheduler from Python to **Go** with **Raft leader election** (pre-vote, check-quorum) and Postgres fencing epochs, cutting CPU **2.4–2.8x** and memory **2.1–2.3x** at unchanged failover latency
- Validated with a custom chaos harness that kills, freezes, and partitions nodes under **25 jobs/s**, losing **0 of 3,321 jobs** with **exactly-once effects**, and surfaced a latent queue-wedging bug in the original scheduler during the port
- Tested Go with **-race** and goleak at **92.5% coverage** plus Raft partition simulations and envtest operator suites (**87.5%**); instrumented Prometheus/Grafana and OpenTelemetry, with Terraform for EKS/GKE

**Real-Time Collaborative Code Review Platform | Rust, Python, React, TS, Redis, GCP, K8s**

- Replaced a Python WebSocket relay with a **Rust** real-time server (tokio, axum, yrs CRDT) using one actor per document, with server-side fan-out under **0.5 ms** for **2,929 of 2,930** updates at 200 connections
- Scaled across pods with document-affinity ingress routing and per-document **Redis Streams**, batching updates into Postgres with snapshots, compaction, and crash recovery from the stream
- Added backpressure via bounded per-connection queues and cursor coalescing, graceful SIGTERM drain with jittered reconnects, and **KEDA** autoscaling on active connections
- Verified with **80+ Rust tests** (proptest CRDT convergence, testcontainers multi-node failover); FastAPI issues single-use, document-bound WebSocket tickets with role-based access

**Data Processor | Python, FastAPI, GCP, Terraform, Docker, GitHub Actions, Pub/Sub**

- Architected serverless platform handling **2,133 requests/min** with **1,169ms p95 latency**, processing **3M+ monthly requests** at **60% lower cost** than VM alternative
- Deployed async Pub/Sub architecture achieving **95% success rate** and **zero message loss** through automatic NACK-based retries, dead-letter queues, and exponential backoff
- Designed multi-tenant security using Firestore sub-collections with physical data isolation and PII redaction, supporting **50+ concurrent tenants**
- Orchestrated full deployment lifecycle using Test-Driven Development, with Terraform managing **15+ GCP resources** and GitHub Actions CI/CD running **21 PyTest suites** at **85% coverage**

## TECHNICAL SKILLS

- **Languages**: Python, Go, Rust, Java, JavaScript, TypeScript, HTML, CSS, SCSS, SQL, Shell, YAML, HCL
- **Frameworks & Tools**: Flask, FastAPI, React, Angular, Tokio, Axum, controller-runtime, Power BI, Tableau
- **Databases:** MSSQL, PostgreSQL, SQLite, MongoDB, FireStore, Redis
- **DevOps**: CI/CD, Kubernetes, KEDA, Docker, GitHub Actions, Terraform, Packer, AWS, Azure, GCP, Linux